Sesiunea 6. Informații invizibile: forensics și steganografie

suport de curs · 2 ore · Marius Dumitrașcu (marius@cybercor.org)

DESPRE SESIUNE. Ce prinzi, pe rețea sau într-un fișier, rar este în clar. De obicei este un strat de codare, un XOR cu o cheie scurtă, un hash de parolă, sau un mesaj pitit într-o poză. Astăzi învățăm să citim ce nu se vede, programatic, în Python. Firul are cinci verbe: desfaci straturi de codare, spargi o cheie scurtă, recunoști și ataci un hash, citești urmele pe care le lasă un fișier (semnătură, text, metadate), și dezvălui ce a fost ascuns la vedere într-o imagine. Scopul real rămâne limbajul: lucrul cu octeți (bytes), operatorii pe biți, bibliotecile hashlib și Pillow, expresiile regulate pe octeți, o funcție și un test. Forensics-ul este terenul pe care exersăm.

O NOTĂ DESPRE NIVEL. Această sesiune presupune ce ai scris până acum: deschiderea fișierelor, dicționare, bucle, funcții, argparse, un test. Nu reluăm acele baze pas cu pas. Ne concentrăm pe ce este cu adevărat nou: diferența dintre text și octeți (str și bytes), operatorul XOR pe octeți, și ideea că un fișier este, la bază, un șir de numere pe care îl citim cum vrem.

ȚINTE PERMISE ȘI CADRUL LEGAL. Toate exercițiile folosesc fișiere pregătite pentru laborator (generate de tine sau date de profesor) și propriile tale fișiere. Forensics pe dispozitivul, contul sau fișierele altcuiva, fără acordul lui sau un mandat, este acces neautorizat la date, o infracțiune (Cod penal RM, capitolul XI). Aceleași unelte, pe ce îți aparține sau pe ce ai voie să analizezi, sunt muncă legală. Diferența nu este unealta, ci permisiunea.

UNDE SUNTEM PE SCARA DE PROGRAMARE. Cursul se numește Tehnici de programare: scopul real este să stăpânești Python ca pachet complet, până la obiecte. Fiecare sesiune urcă o treaptă. Ai pus deja variabilele, tipurile, controlul (if, for) și, săptămâna trecută, lucrul cu rețeaua și web-ul. Treapta de azi este lucrul cu date binare: octeți (bytes vs str) și operatori pe biți, temelia pentru tot ce atinge fișiere. Urmează colecțiile, funcțiile și excepțiile, generatoarele, apoi obiectele și clasele. Povestea de securitate rămâne motorul; ce măsurăm este programarea.

Conceptul zilei: octeți și biți (date binare)

Înainte de orice decodare, o distincție de care atârnă toată sesiunea. În Python, textul pentru oameni este de tip str; ce stă, de fapt, pe disc sau pe fir este de tip bytes, un șir de octeți, adică de numere între 0 și 255. Aproape orice eroare de început în forensics vine din amestecarea celor două.

```
s = "Salut"        # str: text, pentru oameni
b = b"Salut"       # bytes: octeti bruti

s.encode()         # str   -> bytes
b.decode()         # bytes -> str

b[0]               # 83  (un octet indexat = un NUMAR)
bytes([83, 97])    # b"Sa"  (numere -> octeti)
```

Reține regula practică: citești fișiere cu open(cale, "rb"), adică în mod binar, și primești octeți. Treci la text cu .decode() doar când ești sigur că acei octeți chiar reprezintă text. Când indexezi un obiect bytes, nu primești un caracter, ci numărul acelui octet.

O punte: un octet este un număr

În C, legătura este și mai directă. Tipul char este literalmente un octet, deci un caracter și un număr sunt același lucru, afișat cum alegi: ca număr, ca literă, sau ca hexazecimal.

```
/* in C: acelasi octet, trei haine */
```

```
unsigned char b = 'S';
printf("%d\n", b);    /* 83  */
printf("%c\n", b);    /* S   */
printf("%02x\n", b);  /* 53 (hex) */
```

În Python, b[0] îți dă direct numărul 83; în C, caracterul și numărul sunt deja unul și același. Un octet nu „este" literă sau număr: este o valoare între 0 și 255, pe care noi o citim cum avem nevoie. Exact asta facem azi cu fișierele.

Operația	Ce face	Se desface?
codare (base64, hex, URL)	altă formă, aceiași octeți	da, oricine, fără cheie
criptare (AES, XOR)	ascunde conținutul cu o cheie	doar cu cheia
hash (md5, sha256)	o amprentă, o singură direcție	nu: doar ghicești intrarea

Prima întrebare în fața unui șir necunoscut: este doar recodat, este criptat, sau este un hash? Răspunsul îți spune ce unealtă să folosești. Confuzia dintre cele trei este cea mai des întâlnită greșeală.

Partea I. Straturi: encoding și cum îl recunoști

O codare schimbă forma octeților, nu îi ascunde. base64, hexazecimalul și URL-encoding sunt cele trei pe care le vezi peste tot. Cheia nu este să le încerci la nimereală, ci să recunoști stratul după forma lui.

base64: stratul cel mai des întâlnit

```
import base64
s = "U2FsdXQgbHVtZQ=="
b = base64.b64decode(s)
print(b)            # b"Salut lume"
print(b.decode())   # Salut lume
```

Sintaxă:

- `base64.b64decode(s)` desface base64 și întoarce octeți (bytes).
- `.decode()` transformă octeții în text, dacă sunt text.
- `==` la coadă umplutura base64: un indiciu vizual puternic.

Tehnică: base64 reprezintă octeți folosind 64 de caractere citibile (A-Z, a-z, 0-9, plus „+" și „/"). Nu are cheie, deci nu ascunde nimic: doar împachetează octeți ca text imprimabil.

Funcționalitate: Apare în cookie-uri, token-uri, fișiere de configurare, atașamente de e-mail. Un analist îl recunoaște din prima după alfabet și după semnele „=" de la final, și îl dă jos într-o linie.

Recunoașterea stratului: te uiți la alfabet

```
# hex:    doar 0-9 a-f, lungime para
# base64: A-Za-z0-9+/ , poate termina cu =
# URL:    contine %xx  (ex. %2F = / )

s = "4861 7368"   # doar cifre si a-f -> hex
```

Sintaxă: Nu există o funcție care îți spune sigur ce strat e; te uiți la mulțimea de caractere care apar.

Tehnică: Fiecare codare are un alfabet propriu: hexul folosește 16 simboluri, base64 folosește 64 plus „=", URL-encoding conține „%". Citind forma, restrângi ce să încerci.

Funcționalitate: Aceasta este prima abilitate reală de forensics: nu încerci toate decodările orbește, ci pornești de la cea mai probabilă, judecând după alfabet, lungime și umplutură.

Hexazecimal: octeți scriși pe cifre

```
h = "48656c6c6f"
b = bytes.fromhex(h)
print(b)        # b"Hello"
print(b.hex())  # 48656c6c6f
```

Sintaxă:

- `bytes.fromhex(h)` citește șirul hex și întoarce octeții.
- `b.hex()` drumul invers: octeți -> șir hexazecimal.

Tehnică: Hexul scrie fiecare octet cu două cifre, de la 00 la ff. Este modul universal de a arăta octeți bruți.

Funcționalitate: Îl vei întâlni în editoare hex, în dump-uri de memorie și în semnăturile de fișier din partea a patra. Perechea fromhex / .hex() te mută între cele două reprezentări.

URL-encoding: de ce apare %2F în adrese

```
from urllib.parse import unquote, quote
u = "%2Fetc%2Fpasswd"
print(unquote(u))          # /etc/passwd
print(quote("/etc/passwd"))  # %2Fetc%2Fpasswd
```

Sintaxă:

- `unquote(u)` readuce caracterele %xx la normal.
- `quote(...)` codează caracterele speciale ca %xx.
- `%2F, %20, %00` sunt „/", spațiu, respectiv octetul nul.

Tehnică: URL-encoding (procent-encoding) scrie caracterele care nu sunt permise într-o adresă sub forma %xx, unde xx este codul hexazecimal al octetului.

Funcționalitate: Îl citești des în loguri de web, iar în atac îl folosești ca să treci un payload printr-un filtru: o cale de tip „../" poate ajunge la server codată ca „..%2F".

Straturi multiple: când decodarea dă tot zgomot

```
s = "NjE3MzYzNzU2ZTcz"
pas1 = base64.b64decode(s)
print(pas1)   # b"617363756e73"  <- doar 0-9 a-f = hex
pas2 = bytes.fromhex(pas1.decode())
print(pas2)   # b"ascuns"  <- text, te opresti
```

Sintaxă:

- `rezultatul unei decodări` poate fi, la rândul lui, o codare.
- `criteriul de oprire` text citibil sau octeți de fișier cunoscut.

Tehnică: Provocările de forensics ascund mesajul sub mai multe straturi. Desfaci unul, te uiți dacă rezultatul este citibil; dacă nu, încerci stratul următor.

Funcționalitate: Scrii o buclă care decodează până iese ceva cu sens. Aici recunoașterea stratului (lecția anterioară) se unește cu automatizarea: fiecare pas alege ce să încerce după forma rezultatului.

Partea II. Cheia: XOR și spargerea lui

base64 nu are cheie. XOR are, și este cea mai simplă formă de criptare. Dar dacă cheia este un singur octet, există doar 256 de variante, deci le putem încerca pe toate.

XOR: operația care se anulează pe ea însăși

```
a = 0x41          # litera A
k = 0x10          # cheia
c = a ^ k         # 0x51 = Q  (criptat)
print(c ^ k)      # 0x41 = A  (la loc)
# proprietatea cheie:  x ^ k ^ k == x
```

Sintaxă:

- `^` operatorul XOR: compară bit cu bit.
- `x ^ k ^ k == x` același XOR cu aceeași cheie te duce înapoi.

Tehnică: XOR dă 1 pe pozițiile unde biții diferă. Proprietatea lui esențială este simetria: aplicat de două ori cu aceeași cheie, revine la valoarea de start. Deci criptezi și decriptezi cu aceeași operație.

Funcționalitate: Apare des în malware și în provocări tocmai pentru că este simplu și reversibil. Securitatea lui stă doar în cheie; o cheie scurtă nu rezistă.

XOR pe un întreg mesaj

```
date = bytes.fromhex("3223302d2e23")
cheie = 0x42
clar = bytes(b ^ cheie for b in date)
print(clar)          # b"parola"
print(clar.decode()) # parola
```

Sintaxă:

- `b ^ cheie for b in date` treci fiecare octet prin XOR cu cheia.
- `bytes(...)` aduni rezultatele într-un nou șir de octeți.
- `errors="replace"` nu crapă dacă un octet nu e text valid.

Tehnică: Parcurgi octet cu octet și construiești un nou obiect bytes. Dacă rezultatul arată a text (litere, spații, punctuație), ai nimerit cheia; dacă e tot zgomot, nu este cheia potrivită.

Funcționalitate: Acest criteriu („arată a text") este exact ce ne permite să spargem cheia automat: încercăm toate valorile și o alegem pe cea care produce text.

Spargerea prin forță brută: doar 256 variante

```
for k in range(256):
    clar = bytes(b ^ k for b in date)
    if all(32 <= c < 127 for c in clar):
        print(k, clar)
```

Sintaxă:

- `range(256)` toate valorile posibile ale unei chei de un octet.
- `all(32 <= c < 127 ...)` păstrezi doar rezultatele complet imprimabile.
- `32-126` intervalul ASCII imprimabil: un filtru simplu.

Tehnică: Nu spargi nimic complicat: lași calculatorul să încerce toate cheile și filtrezi variantele care arată a limbă. Filtrul poate fi oricât de fin (mai târziu: „conține cuvinte dintr-un dicționar").

Funcționalitate: Este tiparul general al forensics-ului: generezi toate posibilitățile, apoi filtrezi după un criteriu de plauzibilitate. Îl vei reîntâlni la spargerea de hash-uri.

> **XOR, VĂZUT ÎN C.** În C, aceeași operație se scrie buf[i] ^= key: aplică XOR și rescrie octetul pe loc. Este exact bucla din Python, strânsă într-o linie, dar pe memoria fișierului direct. Multe programe rău intenționate își

> ascund șirurile cu un XOR scurt la pornire, tocmai pentru că este ieftin și păcălește o scanare naivă. De aceea analistul caută chei scurte.

Partea III. Amprente: hashing și spargerea lui

Spre deosebire de codare și criptare, un hash nu se inversează. Este o amprentă de lungime fixă a unei intrări. Nu îl „decriptezi": ca să afli intrarea, trebuie să o ghicești.

hashlib: amprenta unui mesaj

```
import hashlib
h = hashlib.sha256(b"parola").hexdigest()
print(h)   # a80b568a...  (64 de caractere)
# schimbi un octet -> amprenta total diferita
```

Sintaxă:

- `hashlib.sha256(bytes)` calculează amprenta sha256 a octeților.
- `.hexdigest()` o arată ca șir hexazecimal.

Tehnică: O funcție de hash ia octeți și produce o amprentă de lungime fixă. Este deterministă (aceeași intrare dă mereu aceeași ieșire), dar o schimbare cât de mică a intrării schimbă complet ieșirea.

Funcționalitate: De aceea serverele nu păstrează parola, ci hash-ul ei: la autentificare compară amprentele. Nu pot citi parola înapoi, dar, din același motiv, nici tu direct.

Identificarea: citești hash-ul după lungime

```
# md5    = 32 caractere hex
# sha1   = 40 caractere hex
# sha256 = 64 caractere hex
h = "5f4dcc3b5aa765d61d8327deb882cf99"
print(len(h))   # 32 -> probabil md5
```

Sintaxă:

- `len(h)` numeri caracterele hex ale amprentei.
- `32 / 40 / 64` lungimea restrânge ce algoritm este.

Tehnică: Fiecare algoritm produce o amprentă de lungime fixă. Numărând caracterele, restrângi ce algoritm să presupui. Nu este infailibil (32 de caractere hex pot fi și altceva), dar este punctul de plecare.

Funcționalitate: Este prima mișcare în fața unui hash necunoscut, fiindcă îți spune ce unealtă de spargere să pornești. Unelte precum hashid automatizează exact această judecată.

Spargerea prin dicționar

```
tinta = "5f4dcc3b5aa765d61d8327deb882cf99"
for cuv in open("rockyou.txt", encoding="latin1"):
    cuv = cuv.strip()
    if hashlib.md5(cuv.encode()).hexdigest() == tinta:
        print("gasit:", cuv)   # password
        break
```

Sintaxă:

- `open("rockyou.txt")` lista de parole scurse, folosită la testare.
- `.strip()` scoate sfârșitul de linie din fiecare cuvânt.
- `== tinta` compari amprenta candidatului cu ținta.

Tehnică: Nu inversezi hash-ul: hashuiești fiecare cuvânt dintr-o listă și compari cu ținta. Când o amprentă coincide, ai aflat intrarea. Puterea atacului stă în listă, nu în algoritm.

Funcționalitate: O parolă din primele o mie cele mai folosite cade în milisecunde. De aici regula de apărare: parole lungi, unice, care nu apar în dicționare.

Apărarea: de ce „sarea" strică atacul

```
import os, hashlib
sare = os.urandom(16)   # aleatorie, per utilizator
h = hashlib.sha256(sare + b"parola").hexdigest()
# doi useri cu aceeasi parola -> hash DIFERIT
```

Sintaxă:

- `os.urandom(16)` șir aleatoriu de 16 octeți.
- `sare + b"parola"` adaugi sarea înainte de a calcula hash-ul.

Tehnică: Sarea (salt) este un șir aleatoriu, unic pentru fiecare utilizator, adăugat la parolă înainte de hash. Nu este secretă; se păstrează lângă hash.

Funcționalitate: Efectul: aceeași parolă dă amprente diferite la doi oameni. Atacatorul nu mai poate sparge o dată pentru toți, iar tabelele gata calculate (rainbow tables) devin inutile. În practică se folosesc funcții lente anume concepute (bcrypt, argon2), dar principiul sării este fundamentul.

HMAC: amprentă cu cheie, pentru integritate

```
import hmac, hashlib
cheie = b"secret-comun"
mesaj = b"transfera 100 lei"
t = hmac.new(cheie, mesaj, hashlib.sha256).hexdigest()
# trimiti mesaj + t; fara cheie, t nu se poate reface
```

Sintaxă:

- `hmac.new(cheie, mesaj, alg)` calculează o etichetă legată de o cheie.
- `.hexdigest()` o arată ca șir hex, ca la un hash.

Tehnică: Un hash simplu spune doar „mesajul nu s-a stricat pe drum", dar oricine îl poate reface. HMAC leagă amprenta de o cheie secretă: doar cine o are poate produce sau verifica eticheta.

Funcționalitate: Apare la semnarea cererilor către API-uri, la token-urile JWT și la webhook-uri. Îl întâlnești ca garanție că un mesaj vine de la cine știe cheia și nu a fost modificat.

Partea IV. Urme: ce spune un fișier despre el

Extensia unui fișier este doar o etichetă, ușor de schimbat. Adevărul stă în octeți: în primii octeți (semnătura), în textul citibil pe care îl conține, și în metadatele pe care le poartă.

Magic bytes: primii octeți spun tipul real

```
with open("misterios.dat", "rb") as f:
    cap = f.read(4)
print(cap)   # b"%PDF"  -> este un PDF
# chiar daca se numeste .dat sau .jpg
```

Sintaxă:

- `open(..., "rb")` deschizi în mod binar: octeți neatinși.
- `f.read(4)` citești primii patru octeți.
- `b"%PDF"` semnătura unui PDF (magic bytes).

Tehnică: Fiecare format începe cu o semnătură fixă. Citind primii câțiva octeți, recunoști tipul real, indiferent de extensie. Comanda file din Linux face exact această verificare.

Funcționalitate: Este prima mișcare în fața unui fișier necunoscut: îi citești capul. O poză redenumită în „.txt" sau un executabil deghizat în „.jpg" se dau de gol imediat.

Semnături de știut pe de rost:

Format	Primii octeți (hex)	Ca text
PNG	`89 50 4E 47`	.PNG
JPEG	`FF D8 FF`	(nu e text)
PDF	`25 50 44 46`	%PDF
ZIP, docx, jar	`50 4B 03 04`	PK..
ELF (binar Linux)	`7F 45 4C 46`	.ELF

Un ZIP și un fișier .docx au aceeași semnătură „PK", fiindcă un docx chiar este o arhivă ZIP. Deschide-l ca arhivă și îi vezi conținutul.

strings: scoți textul citibil dintr-un binar

```
import re
date = open("program.bin", "rb").read()
# siruri de 4+ caractere imprimabile:
for m in re.findall(rb"[ -~]{4,}", date):
    print(m.decode())
```

Sintaxă:

- `rb"..."` o expresie regulată pe OCTEȚI (prefixul rb).
- `[ -~]` intervalul de octeți de la spațiu (0x20) la „~" (0x7e).
- `{4,}` cel puțin patru la rând.

Tehnică: Operația „strings" caută secvențe de caractere imprimabile într-un fișier binar. Observă prefixul rb: expresia lucrează pe octeți, nu pe text, fiindcă fișierul este binar.

Funcționalitate: Este prima unealtă pe orice binar necunoscut. Un program rău intenționat îți scapă adesea adresa serverului de comandă, o cale sau un mesaj, direct în text.

> **DE CE „rb", VĂZUT ÎN C.** În C, deschiderea cu fopen(cale, "rb") înseamnă „read binary": iei octeții exact cum sunt. Fără „b", unele sisteme „traduc" sfârșitul de linie (pe Windows, CRLF în LF) și strică un fișier binar. fread umple un buffer cu octeți bruți, exact ca f.read() în Python. Regula: orice nu este sigur text se deschide cu „rb".

Metadate EXIF: ce ascunde o fotografie

```
from PIL import Image
from PIL.ExifTags import TAGS
img = Image.open("poza.jpg")
exif = img._getexif() or {}
for id, val in exif.items():
    print(TAGS.get(id, id), "=", val)
```

Sintaxă:

- `Image.open(...)` deschide imaginea cu biblioteca Pillow.
- `img._getexif()` citește blocul de metadate EXIF.
- `TAGS.get(id, id)` traduce id-ul numeric în nume citibil.

Tehnică: EXIF este un bloc de metadate pe care aparatul îl pune în poză: model, dată, setări, și adesea coordonate GPS. Pillow îl citește, iar TAGS traduce numerele în nume.

Funcționalitate: OSINT-ul trăiește din asta: o singură poză publică poate trăda unde locuiește cineva. Apărarea este simplă: șterge metadatele înainte de a publica (unealta exiftool o face pentru orice format).

GPS: din poză pe hartă

```
gps = exif.get(34853, {})   # blocul GPS
# latitudine/longitudine ca
# (grade, minute, secunde) + emisfera (N/S, E/V)
print(gps)
```

Sintaxă:

- `exif.get(34853, {})` citești blocul GPS, sau un dict gol dacă lipsește.
- `grade, minute, secunde` formatul coordonatei în EXIF.

Tehnică: Blocul GPS din EXIF ține latitudinea și longitudinea, în grade, minute și secunde, plus emisfera. Le convertești în grade zecimale și le pui pe o hartă.

Funcționalitate: Un detector de GPS în poze este o unealtă de câteva linii, dar arată cât de mult publică oamenii fără să știe. De aceea rețelele serioase șterg EXIF la încărcare, iar tu ar trebui s-o faci oricum.

File carving: scoți un fișier ascuns în altul

```
date = open("suspect.png", "rb").read()
# un ZIP incepe cu "PK\x03\x04":
i = date.find(b"PK\x03\x04")
if i != -1:
    open("ascuns.zip", "wb").write(date[i:])
```

Sintaxă:

- `date.find(b"PK\x03\x04")` caută semnătura ZIP în octeți.
- `date[i:]` feliezi de la semnătură până la sfârșit.
- `open(..., "wb")` scrii octeții extrași într-un fișier nou.

Tehnică: Mai multe fișiere pot sta lipite într-unul singur. Cauți semnătura celui de-al doilea și tai de acolo încolo. Unealta binwalk automatizează această căutare pentru toate semnăturile cunoscute.

Funcționalitate: Este un clasic de steganografie: o poză care, de la un punct, conține de fapt o arhivă. Vizualizatorul de imagini se oprește la sfârșitul pozei și ignoră restul, dar octeții rămân acolo.

Partea V. Ascuns la vedere: steganografie și produsul

Criptarea ascunde conținutul unui mesaj; steganografia ascunde însuși faptul că există un mesaj. Cea mai simplă tehnică pe imagini este LSB: folosești bitul cel mai puțin semnificativ al fiecărui pixel.

Ideea LSB: bitul care nu se vede

```
# un canal de culoare = un octet, 0..255
# ultimul bit schimba valoarea cu 1 (ex. 100 vs 101)
# ochiul NU vede diferenta
# dar acolo incape un bit de mesaj
```

Sintaxă: LSB vine de la „Least Significant Bit", bitul cel mai puțin semnificativ al unui octet.

Tehnică: O culoare este un octet între 0 și 255. Schimbarea ultimului ei bit mută valoarea cu o unitate, imperceptibil pentru ochi, dar suficient ca să ascunzi acolo un bit de informație.

Funcționalitate: Capacitate mare și invizibil vizual: de aceea LSB este baza steganografiei pe imagini. În schimb, lasă o urmă statistică, pe care o folosim la detecție.

Ascunzi un mesaj în ultimii biți

```
from PIL import Image
img = Image.open("gazda.png").convert("RGB")
px = list(img.getdata())   # [(r, g, b), ...]
biti = "".join(f"{b:08b}" for b in b"secret")
# pui cate un bit in ultimul bit al fiecarui canal:
#   r = (r & ~1) | bit
```

Sintaxă:

- `f"{b:08b}"` scrie un octet ca 8 biți (text de 0 și 1).
- `r & ~1` pune ultimul bit pe 0.
- `| bit` scrie în locul lui bitul tău de mesaj.

Tehnică: Transformi mesajul în octeți, apoi în biți, și scrii fiecare bit în ultimul bit al unui canal de culoare. Poza rezultată arată identic cu originalul.

Funcționalitate: Ca să citească cineva mesajul, trebuie să știe regula: câți biți, în ce ordine, unde se termină. Fără ea, informația este practic invizibilă.

Dezvălui mesajul: citești biții înapoi

```
px = list(Image.open("stego.png").convert("RGB").getdata())
biti = ""
for (r, g, b) in px:
    biti += str(r & 1)   # ultimul bit
# regrupezi cate 8 biti -> octeti -> text
```

Sintaxă:

- `r & 1` extrage doar ultimul bit al canalului.
- `regrupezi câte 8` opt biți formează un octet al mesajului.

Tehnică: Dezvăluirea este inversul ascunderii: aceiași pași, citiți invers. Aduni biții, îi grupezi câte opt, îi transformi în octeți și apoi în text.

Funcționalitate: Când apare text citibil, ai găsit mesajul. Când nu, fie nu este stego LSB, fie regula este alta (alt canal, mesaj criptat înainte de ascundere). Aici se vede de ce există unelte dedicate.

Detecția: vezi urma, nu mesajul

```
from collections import Counter
px = list(Image.open("x.png").convert("RGB").getdata())
ultimii = [r & 1 for (r, g, b) in px]
print(Counter(ultimii))   # ~50/50 = suspect
```

Sintaxă:

- `r & 1` ultimul bit al fiecărui pixel.
- `Counter(...)` numără cât de des apare 0 și 1.

Tehnică: Într-o poză naturală, ultimii biți nu sunt perfect echilibrați. Un mesaj ascuns îi face aproape 50/50, ceea ce este un semnal statistic. Counter îți arată distribuția, exact ca la analiza de trafic.

Funcționalitate: Nu „vezi" mesajul, ci urma lui: anomalia. Așa lucrează detecția în securitate în general, căutând ce iese din tipar. Unelte reale: zsteg, stegsolve.

Unelte reale pe care le vei întâlni:

- **binwalk** găsește și extrage fișiere ascunse în alte fișiere.
- **exiftool** citește și șterge metadate, pentru orice format.
- **zsteg / steghide** ascund și scot date din imagini.
- **stegsolve** desface imaginea pe planuri de biți, vizual.
- **strings, file** text citibil și tipul real al fișierului.

Noi le scriem în Python ca să înțelegem mecanismul; în practică folosești și unelte gata făcute. Pe provocările de forensics, reflexul este: file, strings, binwalk, exiftool, apoi stego dacă este o poză.

Produsul: un decoder universal în arsenal

Nu lăsăm tehnicile răzlețe. Le strângem într-un modul din arsenal, un decoder care încearcă singur straturile cunoscute și validează rezultatul, plus un test care îl ține cinstit.

Un decoder care încearcă singur

```
# arsenal/forensics/decode.py
import base64
def incearca(s: str):
    try:
        b = base64.b64decode(s, validate=True)
        if all(32 <= c < 127 for c in b):
            return "base64", b.decode()
    except Exception:
        pass
    # ... la fel pentru hex, apoi XOR brute
    return "necunoscut", s
```

Sintaxă:

- `validate=True` refuză un base64 fals, cu caractere nepermise.
- `all(32 <= c < 127 ...)` confirmă că rezultatul este citibil.
- `return "base64", b.decode()` întoarce stratul și conținutul.

Tehnică: O funcție care încearcă stratul cel mai probabil și verifică dacă rezultatul arată a text. Dacă da, întoarce ce strat a fost și ce a ieșit; dacă nu, trece la următorul.

Funcționalitate: Aceasta este regula produsului: fiecare tehnică de azi devine o funcție reutilizabilă, cu o interfață clară, nu un script de unică folosință. Rezultatul intră direct în raportul care crește.

Un test în care ai încredere

```
# teste/test_decode.py
from arsenal.forensics.decode import incearca
def test_base64():
    tip, txt = incearca("U2FsdXQ=")
    assert tip == "base64"
    assert txt == "Salut"
```

Sintaxă:

- `assert` cere ca ceva să fie adevărat, altfel testul pică.
- `input cunoscut -> output cunoscut` fixezi comportamentul funcției.

Tehnică: Testul cheamă decoderul pe o intrare cunoscută și verifică ieșirea. Dacă mâine strici funcția, testul pică imediat, nu descoperi la examen.

Funcționalitate: O trusă profesională se verifică singură. „passed" este o dovadă obiectivă, spre deosebire de „pare că merge". Asta leagă sesiunea de evaluarea continuă: cod care rulează și este testat.

Recapitulare: azi, pe două coloane

Aceeași muncă, privită din două părți. Pe stânga, ce ai scris; pe dreapta, ce ascunde sau dezvăluie codul, de fapt, în octeții unui fișier.

Programare (cum scrii)	Securitate (cum merge)	Unde
base64, hex, unquote, bytes	recunoști stratul, îl desfaci	straturi
operatorul ^, buclă pe octeți	XOR, spargere prin forță brută	cheia
hashlib, dicționar, hmac	hash, spargere, salt, integritate	amprente
open "rb", regex pe bytes, Pillow	magic bytes, strings, EXIF și GPS	urme
biți, Counter, funcție și test	stego LSB, detecție, arsenal	ascuns

> **CU CE PLECI ASTĂZI.** Reflexul de a citi octeți, nu extensii; întrebarea „ce strat este?" înainte de a decoda orice; și un decoder care, pus în arsenal, desface singur ce prinzi data viitoare. În laboratorul acestei săptămâni construiești decoderul universal cu identificare și spargere, apoi faci forensics pe fișiere și pe o imagine, până la mesajul ascuns, pe fișiere pregătite pentru laborator.